



Extreme Events Explorer

Clustering algorithm documentation





1. Overview

This algorithm detects and tracks spatiotemporal events in gridded climate data by identifying regions that exceed temperature thresholds and maintaining temporal continuity across time slices. Events are defined as contiguous, or close to contiguous, clusters of grid cells that persist for multiple time steps, with spatial proximity determined by Haversine distance calculations.

2. Algorithm Steps

2.1 Preprocessing and Threshold Detection

For each time slice, the algorithm identifies grid cells that meet two conditions:

- The value exceeds (or falls below) the absolute threshold (e.g., 28°C)
- The value exceeds (or falls below) the climatological reference value for that location, where the reference value is defined as the average daily maximum/minimum temperature over the period 1991-2020.

Persistence filtering: To reduce false positives from transient spikes, only cells that meet the threshold conditions for at least 3 consecutive time steps are retained. This is implemented using a rolling window sum over the boolean mask.

2.2 Spatial Clustering

Threshold-exceeding points within each time slice are clustered using spatial proximity. The process:

1. **Distance matrix construction:** For computational efficiency, a sparse distance matrix is built containing only pairwise distances below the neighbour radius (default 200km). Grid-based bounding box search identifies potential neighbours before computing more precise Haversine distances.
2. **Clustering:** A custom DBSCAN variant operates on the sparse distance matrix using breadth-first search. Points within the neighbour radius are grouped into clusters, with a minimum cluster size threshold to filter noise.

The result is a set of spatially-connected *blobs* (clusters of coordinates) for the current time slice.

2.3 Temporal Tracking

Blobs from the current time slice are matched to existing active events using spatial overlap detection:

- For each blob, the algorithm checks if any coordinates overlap with the *previous time slice* of each active event (using a tight epsilon threshold of 1×10^{-6} degrees)
- If overlap is detected, the blob *extends* the event by appending its coordinates and values to the event's time series
- Blobs that don't overlap any active event spawn **new events**

Critical implementation detail: The loop structure ensures each event can be extended by multiple separate blobs, allowing single events to occasionally split into two distinct geographic regions.

Priority is given to larger events when choosing which event a blob should extend



2.4 Event Expiration

After processing each time slice, events that haven't been updated within the expiry window (default 1 day) are considered complete. Valid events (meeting minimum duration of 3 time slices and minimum final size) are finalized and written to output; others are discarded.

Note a key point here: although every pixel is guaranteed to stay above the required threshold for a period of at least 3 consecutive days, it is not guaranteed that it will form part of a cluster on all of those 3 days.

2.5 Event Finalization and Output

When an event expires, comprehensive metadata is generated:

- **Geometric features:** Concave hulls for each time slice, overall event geometry, centroids, and bounding boxes
- **Statistical summaries:** Maximum, mean, and minimum values per time slice and overall; per-pixel maximum values across all time slices
- **Temporal information:** Time series of all slices, duration, start/end dates
- **Land/ocean classification:** Using a land-sea mask, events are classified as land-only, ocean-only, or mixed

Two output files are generated: a compact catalogue entry (JSONL format) for rapid searching, and a complete event file (JSON) containing all time slices and coordinates.

3. Key Parameters

- **threshold:** Absolute temperature threshold (Kelvin)
- **neighbor_radius:** Maximum distance (km) for spatial clustering (used: 250)
- **min_samples:** Minimum cluster size for DBSCAN (used: 30)
- **min_length:** Minimum time slices for valid event (used: 3)
- **expiry_days:** Days without updates before event expires (used: 1)

4. Computational Optimizations

- **Sparse distance matrices:** Only pairwise distances below the neighbour radius are computed and stored, dramatically reducing memory for sparse point distributions
- **Grid-based neighbour search:** Bounding box prefiltering eliminates unnecessary Haversine calculations
- **NumPy vectorization:** Threshold detection and overlap checking use vectorized operations
- **State persistence:** Active events are serialized after each time slice, enabling resumption from interruptions

5. Summary

The algorithm combines threshold-based detection with spatial clustering and temporal tracking to identify coherent spatiotemporal events in gridded climate data. By maintaining event continuity across time slices and applying persistence filters, it effectively distinguishes genuine sustained



events from transient anomalies. The sparse matrix approach and grid-based optimizations enable efficient processing of global high-resolution datasets.



ECMWF - Robert-Schuman-Platz 3, 53175 Bonn, Germany

Contact: <https://support.ecmwf.int/>